1. Arquitectura de infraestructura en Google Cloud

La plataforma MOOC se despliega en el proyecto plasma-sol-513002-k6 de Google Cloud, región us-central1, zona us-central1-a. La Entrega 3 reemplaza las dos máquinas fijas de la Entrega 2 por una arquitectura elástica: un balanceador de aplicaciones externo delante de dos grupos administrados de instancias (MIG) que escalan de forma independiente, una máquina fija y privada para Redis, Cloud SQL y Cloud Storage. Toda la infraestructura está declarada en Terraform y se despliega desde GitHub Actions en cada integración a la rama principal.

Dato	Valor
URL pública	https://136.82.109.164.sslip.io
IP pública del balanceador	136.82.109.164 (global, fija)
Proyecto, región y zona	plasma-sol-513002-k6, us-central1, us-central1-a
Infraestructura como código	deploy/gcp/terraform/ (Terraform, proveedor google)

1.1 Evolución de la Entrega 2 a la Entrega 3

Aspecto	Entrega 2	Entrega 3
Entrada pública	Nginx en la VM web, con IP pública y certificado de Let's Encrypt	Balanceador de aplicaciones externo global con IP fija y certificado administrado por Google
API y frontend	Una VM fija	MIG mig-api, de 1 a 3 instancias
Workers	Una VM fija, con Redis en el mismo host	MIG mig-workers, de 1 a 3 instancias
Redis	Contenedor dentro de la VM del worker	VM fija mooc-redis, sin IP externa, IP 10.0.1.5
VM con IP pública	1 (la web)	Ninguna; solo el balanceador es público
Configuración de las VM	Copiada por SSH en cada despliegue	En los metadatos de la plantilla; cada instancia se configura sola al arrancar

El cambio central es separar el estado del cómputo. La API y los workers no guardan estado local, así que el autoscaler puede crear y borrar sus instancias libremente. Redis, que guarda la cola de trabajos, las sesiones y los límites de tasa, sale de los grupos elásticos y pasa a una máquina propia y estable.

1.2 Justificación de cómputo y máquinas

Todas las máquinas de cómputo son e2-small. La tabla las compara con el límite de cómputo de referencia del curso:

Tabla 1. Perfil e2-small frente al límite de referencia

Recurso	Límite de referencia	e2-small (cada instancia)	¿Cumple?
vCPU	2	2 vCPU compartidas (0,5 sostenida, ráfaga hasta 2)	Sí
Memoria	2 GiB	2 GB	Sí
Disco	30 GiB	30 GB pd-balanced	Sí

Tabla 2. Componentes de cómputo

Componente	Tipo	Instancias	Qué ejecuta	Memoria por contenedor
mig-api	e2-small	1 a 3	API Go (puerto 8080) y frontend Next.js (puerto 3000)	API 768 MiB, frontend 384 MiB
mig-workers	e2-small	1 a 3	Worker asynq con FFmpeg y LibreOffice	1.536 MiB
mooc-redis	e2-small	1 (fija)	Redis 7.4 y Mailpit (SMTP de pruebas)	Redis 1 GiB, Mailpit 128 MiB
Cloud SQL	db-custom-1-3840	1 (administrada)	PostgreSQL 16	3,75 GB

Por qué e2-small. Se eligió e2-small porque coincide con el límite de referencia y es la misma máquina de la Entrega 2. Así, las pruebas de carga se pueden comparar entre entregas sin que cambie el tamaño de cada nodo: lo que cambia es la cantidad de nodos y la forma de repartir el tráfico.

Presupuesto de memoria. Tras el sistema operativo, Docker y el agente de Cloud Monitoring quedan unos 1,6 GB disponibles por máquina. Se reparten con límites de memoria por contenedor, de modo que un pico de un servicio termina en el reinicio de ese contenedor y no en el sistema operativo eligiendo qué proceso matar.

Concurrencia de los workers. Cada worker procesa dos trabajos simultáneos (WORKER_CONCURRENCY=2): dos procesos FFmpeg ya rondan los 2 GB. Para procesar más videos a la vez se agregan instancias, no hilos.

Conexiones a la base de datos. Cada instancia de API abre hasta 20 conexiones y cada worker hasta 5. Con ambos grupos al máximo son 3 × 20 + 3 × 5 = 75 conexiones, por debajo de las 100 que admite la instancia db-custom-1-3840.

1.3 Topología de red y aislamiento

Se usa una red virtual propia (mooc-vpc) en modo personalizado, con una subred por capa:

Tabla 3. Subredes

Subred	Rango	Contiene	IP externa
mooc-api	10.0.0.0/24	Instancias de mig-api	No
mooc-datos	10.0.1.0/24	mooc-redis, con IP interna estática 10.0.1.5	No
mooc-workers	10.0.2.0/24	Instancias de mig-workers	No
Private Service Access	10.30.0.0/20	Cloud SQL (red del proveedor, por peering)	No

Separación de subredes. Cada capa tiene su propio rango, lo que permite escribir el firewall por capa y saber en los registros de flujo de dónde viene cada conexión. Ninguna máquina virtual tiene IP externa: lo único público es la IP del balanceador, que vive en la red perimetral de Google y no dentro de la VPC.

Redis aislado en una VM privada. Redis guarda la cola asynq. Si viviera dentro de un grupo elástico, una reducción del autoscaler podría borrar la instancia que la contiene y con ella los trabajos pendientes. Por eso corre en una VM fija, fuera de los MIG, sin IP externa y con IP interna

estática (10.0.1.5), que es la dirección que la API y los workers reciben en su configuración. Redis escucha solo en esa IP, persiste la cola en disco (appendonly) para sobrevivir a un reinicio, y el firewall solo deja llegar al puerto 6379 a las cuentas de servicio de la API y de los workers.

Conectividad privada hacia Cloud SQL. La instancia de Cloud SQL no tiene IP pública y solo acepta conexiones cifradas. Se alcanza por Private Service Access, mediante un peering con la red de Google. Una regla de salida permite el puerto 5432 solo a la API y a los workers, y otra lo niega a cualquier otra máquina de la red.

Salida a Internet. Private Google Access permite a las máquinas usar Cloud Storage, Artifact Registry, Secret Manager y Cloud Monitoring sin salir a Internet. Cloud NAT, solo de salida, cubre lo que no es de Google: los paquetes que cada instancia nueva instala al arrancar. Sin NAT el autoscaler no podría crear instancias.

Tabla 4. Reglas de firewall (por cuenta de servicio)

Regla	Dirección	Origen	Destino	Puertos
Balanceador y health checks	Entrada	130.211.0.0/22, 35.191.0.0/16	API	TCP 8080, 3000
Redis	Entrada	API y workers	Redis	TCP 6379
SMTP de pruebas	Entrada	API	Redis (Mailpit)	TCP 1025
Administración por IAP	Entrada	35.235.240.0/20	API, workers y Redis	TCP 22
Salida a Cloud SQL	Salida	API y workers	10.30.0.0/20	TCP 5432
Bloqueo a Cloud SQL	Salida	Resto de la VPC	10.30.0.0/20	Todos (deniega)

Las reglas se aplican por cuenta de servicio y no por etiqueta: cambiar la cuenta de servicio de una máquina exige detenerla y un permiso de IAM específico. Todo lo que no aparece en la tabla queda bloqueado por la regla implícita de denegar la entrada. El acceso administrativo por SSH solo entra por el túnel de Identity-Aware Proxy con OS Login, sin el puerto 22 abierto a Internet.

1.4 Configuración del balanceador de carga

Se usa un balanceador de aplicaciones externo global (esquema EXTERNAL_MANAGED). API y frontend quedan bajo el mismo origen, como en la Entrega 2, por lo que la cookie de sesión y la protección CSRF funcionan sin cambios.

Tabla 5. Listeners y reglas de reenvío

Regla de reenvío	IP	Puerto	Destino	Comportamiento
mooc-regla-https	136.82.109.164	443	Proxy HTTPS mooc-proxy-https	Termina TLS y aplica el mapa de URL
mooc-regla-http	136.82.109.164	80	Proxy HTTP mooc-proxy-http	Redirige a https (301)

TLS. El certificado lo emite y renueva Google (Google Trust Services) para 136.82.109.164.sslip.io. La política SSL mooc-tls12 usa el perfil MODERN con TLS 1.2 como versión mínima, y se verificó que una conexión TLS 1.1 es rechazada. El puerto 80 solo redirige porque las cookies de sesión son Secure y no viajarían por http.

Tabla 6. Mapa de URL (mooc-lb)

Prioridad	Coincidencia	Backend service	Timeout
1	.../resources/{recurso}/confirm-upload y .../multipart/complete	mooc-bs-api	600 s
2	Prefijo /api/	mooc-bs-api	60 s
Por defecto	Cualquier otra ruta	mooc-bs-frontend	30 s

Las rutas de prioridad 1 verifican el archivo completo en el servidor (checksum, tipo real y antimalware); con un video grande pueden tardar minutos, por eso tienen un timeout mayor.

Tabla 7. Backend services

Backend service	Grupo	Puerto con nombre	Balanceo	Timeout	Drenaje
mooc-bs-api	mig-api	http-api (8080)	Utilización, máx. 80 %	60 s	30 s
mooc-bs-frontend	mig-api	http-web (3000)	Utilización, máx. 80 %	30 s	30 s

Tabla 8. Health checks

Health check	Uso	Solicitud	Intervalo	Timeout	Sano / no sano
mooc-hc-api	Balanceador (API)	GET /healthz, puerto 8080, HTTP	5 s	3 s	2 éxitos / 2 fallos consecutivos
mooc-hc-frontend	Balanceador (frontend)	GET /, puerto 3000, HTTP	5 s	3 s	2 éxitos / 2 fallos consecutivos
mooc-autohealing-api	Autohealing de mig-api	GET /healthz, puerto 8080, HTTP	15 s	10 s	2 éxitos / 4 fallos consecutivos

Dos health checks distintos. El health check del balanceador es estricto: saca de la rotación en unos 10 segundos una instancia que no responde, pero no la borra. El de autohealing sí la borra y la recrea, por eso es más tolerante y exige un minuto sin respuesta. Con los umbrales del balanceador, una API saturada durante una prueba de carga se recrearía justo cuando más falta hace. La ruta /healthz no se expone al público: el balanceador solo envía /api/ a la API, y los health checks la consultan directamente en el puerto 8080 de cada instancia.

IP real del cliente. El balanceador añade al encabezado X-Forwarded-For la IP del cliente y la suya propia. La API cuenta esos dos saltos desde el final, de modo que el límite de intentos de inicio de sesión usa la IP real del cliente y no una que el cliente pueda inventar.

1.5 Políticas de escalamiento de los grupos administrados

Tabla 9. Parámetros efectivos en GCP

Parámetro	mig-api	mig-workers
Plantilla	mooc-api-* (e2-small, subred mooc-api)	mooc-worker-* (e2-small, subred mooc-workers)
Mínimo de instancias	1	1
Máximo de instancias	3	3
Señal de escalado	Utilización media de CPU	Utilización media de CPU
Umbral objetivo	60 %	60 %

Periodo de inicialización (cooldown)	120 s	120 s
Estabilización de la reducción	~10 min (comportamiento fijo de GCP)	~10 min (comportamiento fijo de GCP)
Tiempo de arranque (warm-up)	~3 a 4 min	~3 a 4 min (imagen más pesada)
Autohealing	Sí, /healthz, retraso inicial de 300 s	No (el worker no expone HTTP)
Actualización de plantilla	Gradual: +1 nueva, 0 no disponibles	Gradual: +1 nueva, 0 no disponibles

Funcionamiento. El autoscaler intenta mantener la CPU media del grupo en 60 %. Si la supera, agrega instancias hasta el máximo; si baja, las retira hasta el mínimo.

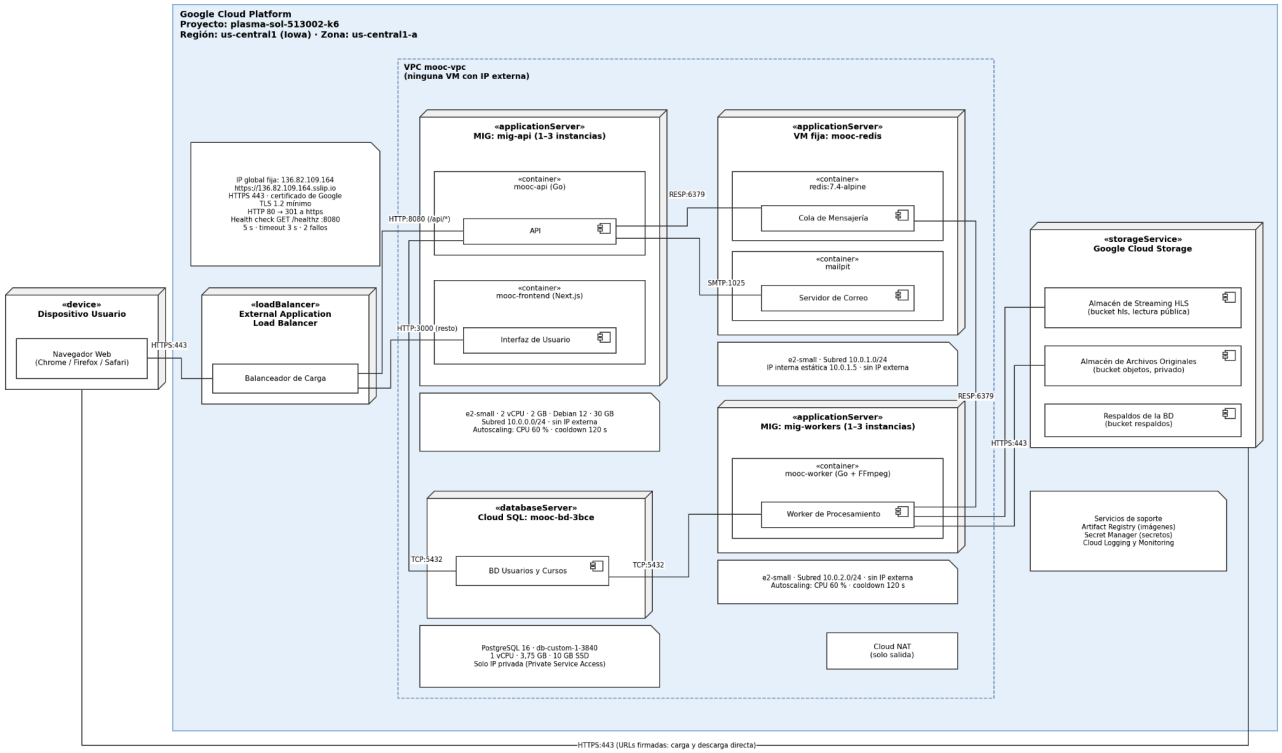
- **Periodo de inicialización (120 s):** el autoscaler no usa la CPU de una instancia recién creada para decidir, porque el arranque la dispara.
- **Estabilización de la reducción:** GCP espera unos 10 minutos de carga baja antes de retirar instancias. Por eso, tras una prueba de carga, las instancias no desaparecen de inmediato.
- **Tiempo de arranque:** una instancia nueva instala Docker, descarga las imágenes y levanta los contenedores antes de atender. Por eso el autohealing no evalúa una instancia nueva durante sus primeros 300 segundos.

Por qué CPU en ambos grupos. En la API, la CPU crece con el número de peticiones. En los workers, FFmpeg satura la CPU mientras transcodifica, así que con la cola llena la CPU es la señal de que hacen falta más workers.

Observación del despliegue. Al desplegar, mig-workers subió a 3 instancias sin trabajos en cola y luego volvió solo a 1. La causa más probable es la CPU del propio arranque (instalar Docker y descargar la imagen con FFmpeg y LibreOffice), que dura más que el periodo de inicialización de 120 s.

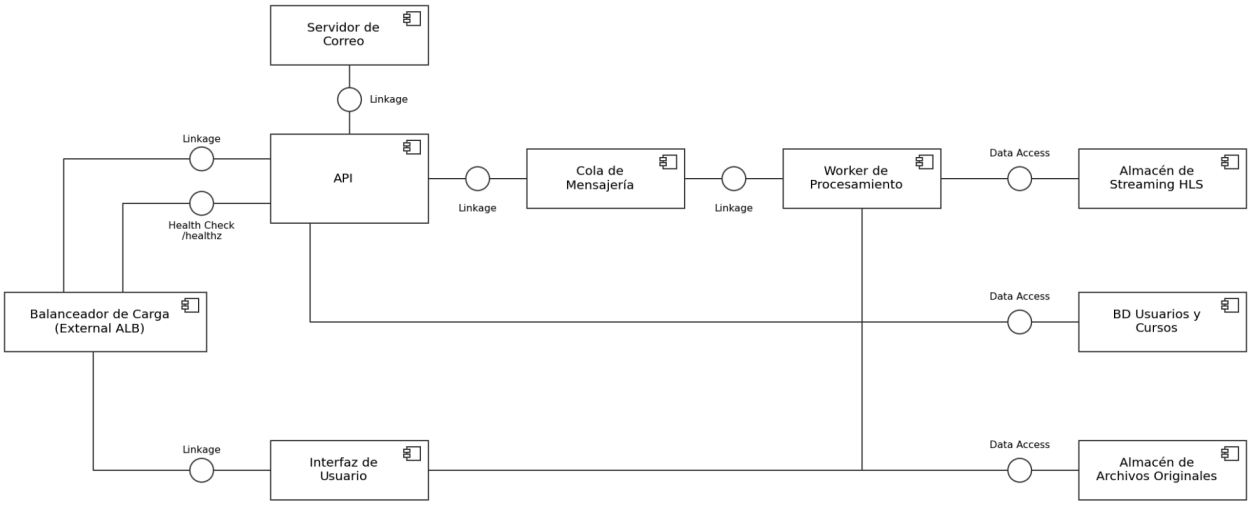
1.6 Diagramas de arquitectura

1.6.1 Diagrama de despliegue



- **Usuario (navegador).** Entra por https://136.82.109.164.sslip.io. Sube y descarga archivos directamente a Cloud Storage con URLs firmadas, sin pasar por el balanceador.
- **Balanceador de aplicaciones externo.** IP global 136.82.109.164. Termina TLS en el puerto 443, redirige el 80 y envía /api/* al puerto 8080 y el resto al 3000 de las instancias de mig-api. Comprueba cada instancia con /healthz.
- **mig-api (subred 10.0.0.0/24).** De 1 a 3 máquinas e2-small, cada una con los contenedores de la API y del frontend.
- **mooc-redis (subred 10.0.1.0/24, IP 10.0.1.5).** Máquina fija con Redis y Mailpit. Recibe conexiones al puerto 6379 desde la API y los workers.
- **mig-workers (subred 10.0.2.0/24).** De 1 a 3 máquinas e2-small con el worker. Toman trabajos de Redis, leen los originales de Cloud Storage y escriben los derivados HLS.
- **Cloud SQL (PostgreSQL 16).** IP privada por Private Service Access. La usan la API y los workers.
- **Cloud Storage.** Bucket objetos (privado: originales, PDF e insignias), bucket hls (lectura pública de los segmentos de video) y bucket respaldos (exportaciones de la base).
- **Cloud NAT.** Solo salida, para instalar paquetes y descargar imágenes.
- **Servicios de soporte.** Artifact Registry (imágenes), Secret Manager (secretos) y Cloud Logging y Monitoring (registros, métricas y autoscaler).

1.6.2 Diagrama de componentes



- **Frontend (Next.js).** Interfaz de estudiantes, profesores y administradores. Llama a la API por REST/JSON bajo el mismo origen.
- **API (monolito modular en Go).** Módulos de identidad, administración, cursos, inscripciones, quizzes, progreso, multimedia e insignias. Expone REST /api/v1 y /healthz. Usa PostgreSQL para los datos; Redis para sesiones, límites de tasa e idempotencia; la cola asynq para publicar trabajos; Cloud Storage para firmar URLs; y SMTP para los correos.
- **Cola asynq en Redis.** Desacopla la API de los trabajos pesados. La API publica y los workers consumen, con reintentos con backoff y cola de mensajes fallidos (DLQ).
- **Worker (Go).** Transcodifica video y audio a HLS con FFmpeg y convierte presentaciones a PDF con LibreOffice. Es idempotente y conserva el archivo original.
- **PostgreSQL.** Fuente de verdad transaccional: usuarios, cursos, versiones, progreso, intentos e insignias.
- **Cloud Storage.** Todos los binarios; ninguno vive en la base relacional.

1.7 Verificación del despliegue

Tabla 10. Verificación sobre plasma-sol-513002-k6

Comprobación	Resultado
La API está sana en el balanceador	HEALTHY (10.0.0.7:8080)
Redis sin IP externa y con su IP fija	mooc-redis en 10.0.1.5, sin IP externa
Las VM en sus subredes	API en 10.0.0.x, workers en 10.0.2.x
El nombre resuelve al balanceador	136.82.109.164.sslip.io → 136.82.109.164
El puerto 80 redirige a https	301 Moved Permanently → https://
El certificado está activo	ACTIVE, emitido por Google Trust Services, válido hasta el 26/01/2027
La página responde por https	200 OK (frontend)
La API responde por el balanceador	/api/v1/health → 200
/healthz no se expone al público	404 por el balanceador
TLS 1.1 rechazado	Sin cifrado negociado
Los MIG vuelven al mínimo en reposo	mig-api 1, mig-workers 1

3. Pruebas de carga y autoscaling

Esta sección mide la plataforma MOOC desplegada en la Entrega 3 con k6 y la compara con la Entrega 2. Responde a tres preguntas: si el balanceador y el grupo de la API eliminan el cuello de botella de la máquina fija, si la API escala sola cuando la CPU supera el 60 %, y si los workers escalan y drenan la cola de transcodificación sin perder trabajos. Cada escenario se ejecutó desde un generador dedicado contra la URL pública del balanceador, y todas las cifras salen de los resúmenes de k6, de las series de Cloud Monitoring y de un observador que registró cada pocos segundos el tamaño de los MIG y la cola de Redis.

Tabla 1. Resumen de resultados de la Entrega 3

Escenario	Qué se probó	Resultado principal
1. Mezcla de flujos web	Cuatro niveles de carga hasta 300 usuarios virtuales sobre catálogo, consumo, quiz y login	207.488 peticiones, 1 fallida (0,0005 %), 196 peticiones por segundo; p95 del catálogo en 38,3 ms. mig-api pasó de 1 a 3 instancias
2. Subida y transcodificación	3 profesores suben videos y 20 estudiantes reproducen HLS	8 de 8 videos listos, 0 % de errores HTTP, p95 de espera en cola de 88,5 s. mig-workers pasó de 1 a 3 instancias
3A. Autoscaling de la API	Ráfaga de 250 usuarios virtuales sobre el catálogo	324.201 peticiones, 0 % de errores, p95 de 729 ms. mig-api: 1, 2, 3 y de vuelta a 1
3B. Autoscaling de workers	Encolado de 126 videos a 20 por minuto	126 de 126 trabajos terminados, 0 duplicados, cola máxima de 79 pendientes. mig-workers: 1, 3 y de vuelta a 1

3.1 Metodología del experimento

3.1.1 Entorno de prueba

Todas las corridas formales usaron la misma infraestructura y el mismo generador, para que lo único que cambie entre escenarios sea la carga aplicada.

Tabla 2. Entorno de las corridas formales

Dato	Valor
Generador de carga	mooc-k6, e2-standard-2, us-central1-a (misma región que la plataforma, VM aparte de la API y los workers)
Objetivo	https://136.82.109.164.sslip.io (balanceador de aplicaciones externo, HTTPS)
Proyecto	plasma-sol-513002-k6
Commit de la plataforma	8dea792 (escenarios 1, 3A y 3B) y eaa924c (escenario 2)
Cloud NAT	Activo (las instancias nuevas instalan paquetes al arrancar)
Concurrencia de workers	WORKER_CONCURRENCY = 2 trabajos simultáneos por worker
Conexiones a la base de datos	DB_MAX_CONNS = 20 por instancia de API
Estado inicial	mig-api y mig-workers en 1 instancia; reposo de 600 s antes de aplicar la carga
Métricas de la plataforma	Cloud Monitoring (CPU, memoria, red y disco por VM; CPU, memoria y conexiones de Cloud SQL; peticiones del balanceador) y el observador del MIG y de la cola asynq

Generador independiente. El generador corre en su propia VM e2-standard-2 y no comparte máquina con ninguna instancia de la plataforma, de modo que k6 no le quita CPU a lo que se mide. En todas las corridas su CPU se mantuvo baja: en el escenario 1 llegó a 0,24 (p95 de 0,21) y en el 3A a 0,44 (p95 de 0,30), muy por debajo del máximo de la máquina, así que el generador no fue el límite de ninguna prueba.

Reposo previo. Cada corrida empieza con 600 s de reposo para confirmar que los grupos están en su mínimo (1 instancia) y que la cola está vacía. Así el escalado que se observa lo provoca la carga de la prueba y no un estado heredado de la anterior.

Observador. Un script registra, en paralelo a k6, el tamaño objetivo y estable de cada MIG y las columnas pending, active, scheduled, retry y archived de cada cola de asynq, junto con la antigüedad del trabajo más viejo. Es la fuente de las líneas de tiempo de esta sección.

3.1.2 Escenarios y parámetros

Tabla 3. Parámetros de cada escenario

Parámetro	Escenario 1	Escenario 2	Escenario 3A	Escenario 3B
Objetivo	Mezcla de flujos web	Subida y transcodificación de video	Saturar la CPU de la API	Saturar la cola de workers
Carga	Niveles de 50, 100, 200 y 300 usuarios virtuales	3 profesores y 20 estudiantes	5 usuarios virtuales de base y pico de 250	Llegada a tasa fija de 20 trabajos por minuto, hasta 8 usuarios virtuales
Rampa	30 s por nivel	No registrada	30 s	20 s
Meseta	3 min por nivel	5 min	3 min de base, 6 min de pico y 15 min de meseta baja	3 min de base, 6 min de pico y 15 min de meseta baja
Contenido	Login, catálogo, consumo y quiz	Videos corto, medio y largo, y audio; reproducción HLS	Catálogo, búsqueda y detalle	Confirmación de subida que encola la transcodificación
Qué escala	mig-api	mig-workers	mig-api	mig-workers

Los escenarios 1 y 2 conservan, según el diseño de la prueba, los usuarios virtuales, las etapas y la duración de la Entrega 2; solo cambia el destino, que ahora es el balanceador. Los escenarios 3A y 3B son nuevos y existen para provocar el escalado.

Por qué el 3B usa llegada a tasa fija. Una versión anterior del 3B encolaba los videos mucho más rápido de lo que los workers podían procesarlos y la cola nunca se drenaba, lo que impedía ver el ciclo completo. Con 20 trabajos por minuto la cola crece lo suficiente para disparar el autoscaler y aun así se vacía dentro de la ventana de observación.

3.1.3 Corridas y evidencias

Tabla 4. Corridas de la Entrega 3

Corrida	Escenario	Inicio y fin (UTC)	Estado
formal-3a-1	3A	2026-10-31 22:42:52 a 23:17:54	Válida
formal-3b-1	3B	2026-10-31 23:32:13 a 23:51:45	Válida
formal-e1-1	1	2026-11-01 00:42:25 a 01:10:04	Válida
formal-e2-1	2	2026-11-01 02:16:26 a 02:33:14	Válida
intento-e2-1-setup-timeout	2	2026-11-01 01:33:45	Descartada: la preparación (setup) superó los 60 s a las 01:44:45 y no se midió carga

Las carpetas piloto-3a, prueba-3b y prueba-3b2 son los ensayos previos con los que se calibró la carga. No se usan en ninguna cifra de esta sección. Se conserva también el intento fallido del escenario 2 junto con su nota, por transparencia.

3.2 Comparativa oficial: Entrega 2 contra Entrega 3

La Entrega 2 corría sobre una sola VM con Nginx. La Entrega 3 pone un balanceador delante de mig-api, que puede crecer hasta 3 instancias e2-small. La tabla compara la misma carga sobre las dos arquitecturas.

Tabla 5. Comparación de los escenarios 1 y 2

Escenario y métrica	Línea base E2 (1 VM Nginx)	Arquitectura E3 (LB + MIGs)	Variación	Análisis causal
1. p95 del catálogo	1.280 ms	38,3 ms	-97 %	El tráfico se reparte entre hasta 3 instancias y desaparece la cola de la máquina única
1. p95 del consumo	2.460 ms	60,0 ms	-98 %	Misma causa: la capa web ya no se satura
1. p95 del quiz	1.626 ms	53,5 ms	-97 %	Misma causa
1. p95 del login	2.459 ms	1.503,8 ms	-39 %	Mejora menor: la mediana es de 353 ms, muy cerca del mínimo de 341 ms, lo que apunta a un costo fijo por petición y no a espera
1. Peticiones por segundo	135	196	+45 %	El sistema atiende más tráfico en el mismo tiempo porque las peticiones ya no esperan
1. Errores HTTP	9,92 %	0,0005 % (1 de 207.488)	-99,99 %	La máquina fija rechazaba o perdía peticiones al saturarse; los grupos repartidos no
2. Errores HTTP	[completar con la tabla oficial de E2]	0 % (0 de 2.073)	[completar]	Con 3 workers los videos se procesan en paralelo y la API solo confirma
2. Videos listos	[completar]	8 de 8	[completar]	Ningún video quedó sin procesar ni falló
2. Espera en cola, p95	[completar]	88,5 s	[completar]	Incluye el arranque de los workers adicionales
2. p95 de latencia HTTP	[completar]	68,9 ms	[completar]	Firma, confirmación y segmentos HLS responden rápido aunque FFmpeg esté ocupado

Notas sobre la comparación. El 9,92 % de errores de la Entrega 2 incluía respuestas 409 que son esperadas por el diseño de la prueba y que entonces no se descontaban, así que parte de esa cifra no era una falla real. Aun corregida, la diferencia con una sola petición fallida de 207.488 sigue siendo clara, pero conviene citar la reducción con esa salvedad. Además, los datos de la Entrega 3 vienen de una sola corrida por escenario, por lo que todavía no se mide la variación entre repeticiones.

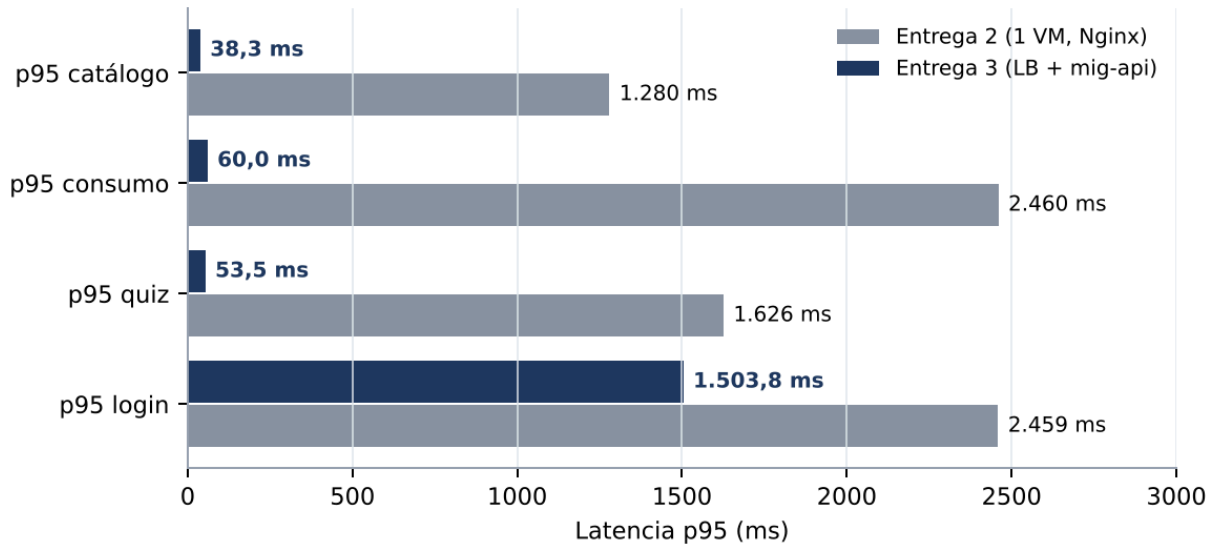


Figura 1. Latencia p95 del escenario 1, Entrega 2 contra Entrega 3

El catálogo, el consumo y el quiz bajan de más de un segundo a menos de 61 ms. El login es la única operación que sigue por encima de 1,5 s en su p95.

3.2.1 Escenario 1: comportamiento durante la corrida

El tráfico del escenario 1 crece por niveles hasta unas 16.700 peticiones por minuto, unas 278 por segundo. La corrida empezó con 1 instancia: el autoscaler pidió la segunda a las 00:53:47 y la tercera a las 00:56:55, y la segunda instancia quedó estable a las 00:57:16 y la tercera a las 01:00:45. Después de terminar la carga, mig-api bajó a 2 a las 01:20:42 y a 1 a las 01:21:25.

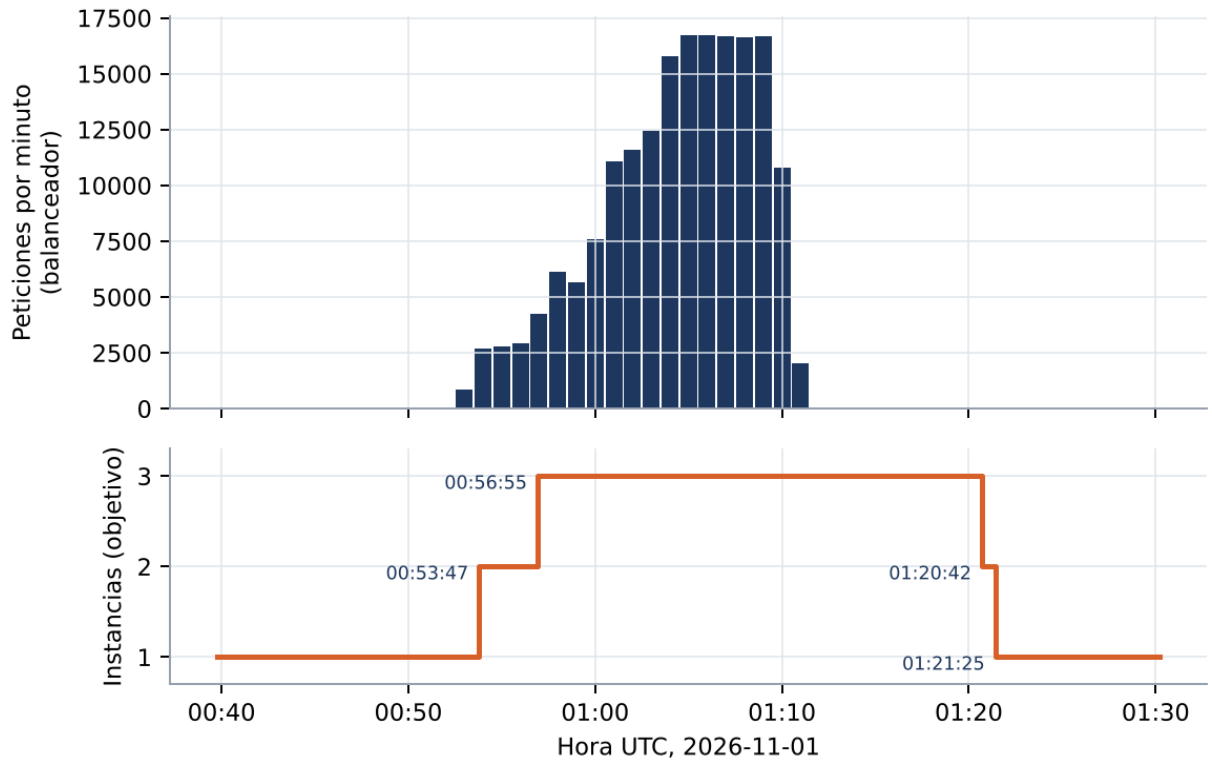


Figura 2. Escenario 1: peticiones por minuto en el balanceador y tamaño de mig-api

Tabla 6. Escenario 1: resumen de k6

Métrica	Valor
Duración de la prueba	1.057 s (17 min 37 s)
Usuarios virtuales máximos	302
Peticiones totales	207.488 (196,3 por segundo)
Peticiones fallidas	1 (0,0005 %)
Comprobaciones (checks)	219.981 correctas, 0 fallidas (100 %)
Latencia de las respuestas esperadas	p50 12,8 ms, p90 26,2 ms, p95 51,3 ms, p99 160,1 ms
Progreso aceptado	37.471 de 37.472 (99,997 %)
Umbrales de k6	Todos cumplidos (errores, p95 y p99 por flujo, checks)

La única petición fallida coincide con el único progreso no aceptado (37.471 de 37.472). La latencia máxima registrada fue de 4.269 ms.

3.2.2 Escenario 2: subida y transcodificación

En este escenario tres profesores suben videos de distinto tamaño (corto, medio, largo y audio) mientras 20 estudiantes reproducen HLS. La subida va directo a Cloud Storage con una URL firmada, así que la API solo firma y confirma, y la transcodificación la hacen los workers con FFmpeg.

Tabla 7. Escenario 2: resultados de k6

Métrica	Valor
Duración de la prueba	2026-11-01 02:16:26 a 02:33:14 UTC (16 min 48 s)
Peticiones HTTP	2.073 (5,08 por segundo), 0 fallidas
Comprobaciones (checks)	1.512 correctas, 0 fallidas (100 %)

Métrica	Valor
Videos que llegaron a estado ready	8 (0 fallidos)
Firma de la URL de subida	p50 18 ms, p95 58,5 ms
Transferencia del archivo	p50 95 ms, p95 125 ms
Confirmación de la subida	p50 113 ms, p95 141 ms
Espera en cola hasta ready	p50 36,2 s, p95 88,5 s, máximo 92,8 s
Tiempo total de subida a ready	p50 36,5 s, p95 88,7 s, máximo 93,0 s
Segmentos HLS	p50 35 ms, p95 70,6 ms, p99 92,9 ms
Latencia HTTP general	p50 26,8 ms, p95 68,9 ms, p99 167,8 ms, máximo 761 ms

La API respondió en milisegundos durante toda la prueba y el tiempo que tarda un video en estar listo lo marca la transcodificación, no la subida. La cola nunca acumuló más de 1 trabajo pendiente, con una antigüedad máxima de 2,6 s.

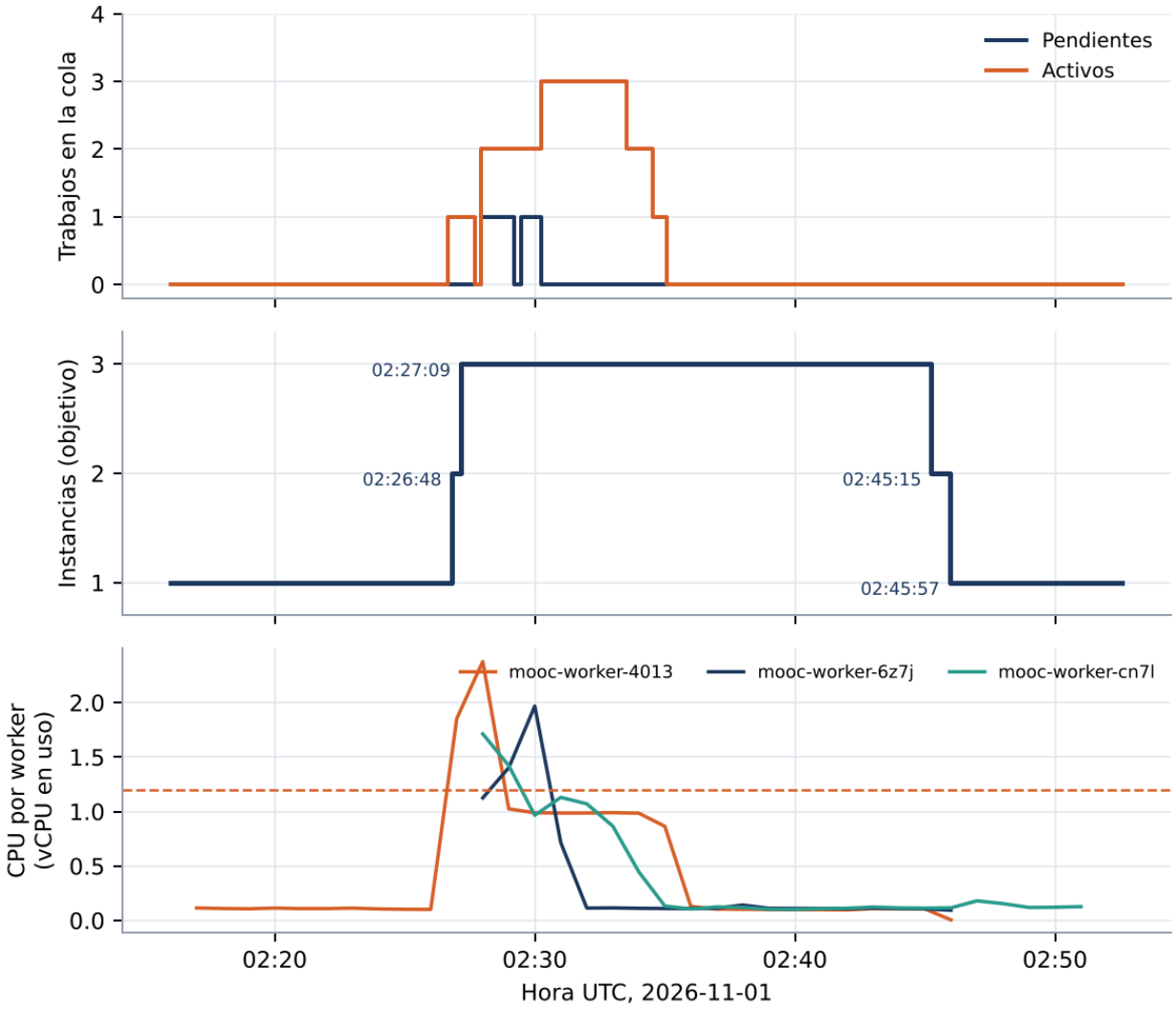


Figura 3. Escenario 2: cola de Redis, tamaño de mig-workers y CPU de cada worker

mig-workers pasó de 1 a 3 instancias a las 02:26:48 y 02:27:09, aunque la cola casi no tuvo trabajos pendientes. La causa es que el autoscaler decide por CPU, y FFmpeg la sube en cuanto empieza a transcodificar. Bajó a 2 a las 02:45:15 y a 1 a las 02:45:57, y quedó estable en 1 a las

02:46:38. Esto sugiere que una señal basada en la profundidad de la cola sería más precisa para los workers.

3.3 Análisis de cuellos de botella

3.3.1 El cuello de la capa web desaparece

En la Entrega 2 todo el tráfico entraba por una VM con Nginx, y esa máquina se saturaba: respondía con latencias de más de un segundo y con errores. En la Entrega 3 el balanceador reparte las peticiones entre las instancias de mig-api y el grupo suma instancias cuando hace falta. El efecto se ve en tres señales: los p95 bajan entre un 97 % y un 98 %, los errores caen a una sola petición en 207.488 y el rendimiento sube de 135 a 196 peticiones por segundo.

3.3.2 La carga se desplaza a Cloud SQL

Con la capa web resuelta, el siguiente recurso compartido es la base de datos. Las instancias de API escalan, pero todas hablan con la misma instancia de Cloud SQL.

Tabla 8. Recursos durante el escenario 1 y el 3A

Recurso	Mediana	p95	Máximo	Lectura
CPU de Cloud SQL, escenario 1	11,8 %	72,9 %	74,8 %	La base pasa de reposo a cerca de tres cuartas partes de su CPU
CPU de Cloud SQL, escenario 3A	12,7 %	69,0 %	71,8 %	Comportamiento parecido bajo la ráfaga
Conexiones a Cloud SQL, escenario 1	0	45	47	Por debajo del tope del pool
Conexiones a Cloud SQL, escenario 3A	2	60	60	Tres instancias por 20 conexiones: el pool llegó a su tope
Memoria de Cloud SQL	30 %	32-35 %	35 %	Sin presión de memoria
CPU de cada instancia de API, escenario 1	0,18 a 0,65	1,00 a 1,04	1,02 a 1,76	Las instancias se calientan solo durante los niveles altos
CPU del generador k6	0,004	0,21	0,24	No limita las pruebas

- **Techo del pool de conexiones.** Cada API abre hasta 20 conexiones, y en el 3A el uso llegó justo a 60 con 3 instancias. Con ambos grupos en su máximo la plataforma puede usar 75 conexiones (3 × 20 de la API y 3 × 5 de los workers), por debajo de las 100 que admite la instancia db-custom-1-3840, pero el margen se agota si se sube el máximo de los MIG.
- **CPU de la base.** Cloud SQL llegó a cerca del 75 % de su CPU con tres instancias de API. Es el primer recurso que habría que ampliar (más CPU o réplicas de lectura) antes de permitir un mayor máximo en mig-api.
- **El login.** Es la operación más lenta y casi no mejoró. Su mediana es de 353 ms y su mínimo de 341 ms, es decir, tarda lo mismo con poca o mucha carga. Esto apunta a un costo fijo por petición (por ejemplo, el hash de la contraseña) y no a la saturación de la infraestructura. Es una hipótesis que falta confirmar con un perfil de la API.

3.4 Escenario 3A: autoscaling de la API

El 3A mantiene 5 usuarios virtuales durante 3 minutos y los sube en 30 s hasta 250, que sostiene 6 minutos sobre catálogo, búsqueda y detalle. Después vuelve a 5 usuarios durante 15 minutos para observar el enfriamiento y la reducción.

Tabla 9. Escenario 3A: línea de tiempo de mig-api

Hora UTC	Evento	Detalle
22:52:53	Inicio de la meseta de base	5 usuarios virtuales, mig-api en 1 instancia
22:55:53	Inicio de la rampa	Sube de 5 a 250 usuarios virtuales en 30 s
22:56:31	Scale-out de 1 a 2	El autoscaler fija el objetivo en 2 instancias
22:56:51	Scale-out de 2 a 3	Fija el objetivo en 3, 20 segundos después
22:59:17	2.ª instancia estable	2 min 46 s después de pedirla
23:00:42	3.ª instancia estable	3 min 51 s después de pedirla; tres réplicas repartiendo carga
23:02:23	Fin de la ráfaga	Vuelve a 5 usuarios virtuales
23:13:54	Scale-in de 3 a 2	11 min después de terminar la ráfaga
23:23:57	Scale-in de 2 a 1	10 min después de la primera bajada
23:26:04	mig-api estable en 1	Fin del ciclo

Tabla 10. Escenario 3A: resultados de k6

Métrica	Valor
Peticiones totales	324.201 (216,0 por segundo)
Peticiones fallidas	0 (0 %)
Comprobaciones	324.201 correctas (catálogo, búsqueda y detalle responden 200)
Latencia	p50 17,7 ms, p90 218,3 ms, p95 728,9 ms, p99 978,5 ms, máximo 3.209 ms
Usuarios virtuales máximos	250
Pico del balanceador	51.230 peticiones por minuto, unas 854 por segundo
CPU máxima de una instancia de API	2,02 (mooc-api-1j95), la primera instancia, en el arranque de la ráfaga
Conexiones máximas a Cloud SQL	60 (3 instancias por 20)
CPU máxima de Cloud SQL	71,8 %

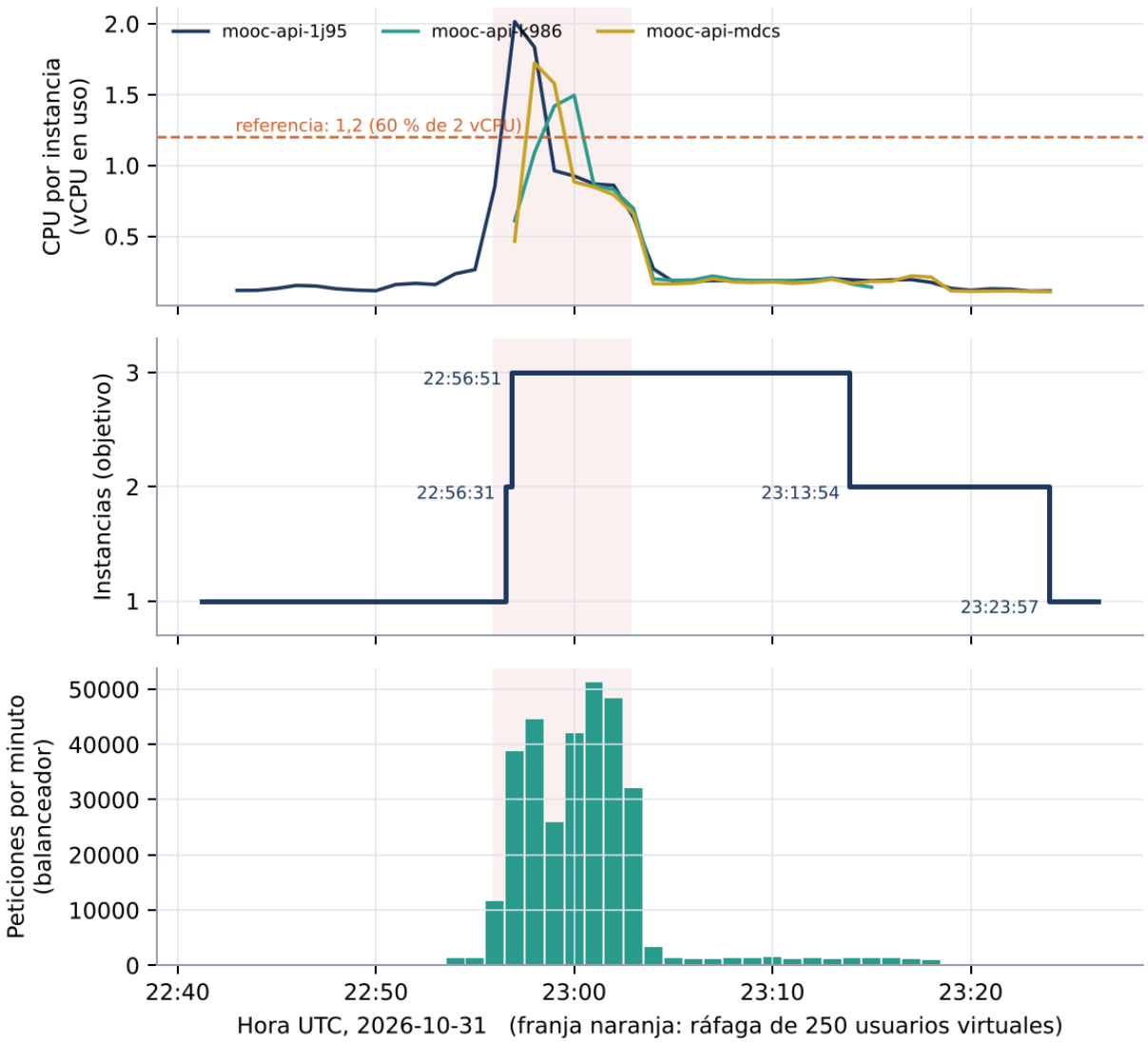


Figura 4. Escenario 3A: CPU de las instancias de API, tamaño de mig-api y peticiones del balanceador

La CPU de la primera instancia llega a 2,02 durante la ráfaga y las instancias nuevas aparecen con CPU alta mientras arrancan. Las tres bajan juntas cuando termina la ráfaga. El tráfico del balanceador pasa de unas 1.300 a más de 51.000 peticiones por minuto, y todas se respondieron con 200.

Cómo leer el p95 de 729 ms. Durante los primeros 3 a 4 minutos de la ráfaga la API trabajó con menos réplicas de las necesarias, porque una instancia nueva tarda en instalar Docker, bajar las imágenes y levantar los contenedores. Ese periodo probablemente explica la cola de latencia (p95 y p99 cercanos a 1 s), no una falla del balanceador: la mediana de toda la corrida fue de 17,7 ms.

Unidad de las series de CPU. Las series de CPU por VM se exportaron tal como las entrega Cloud Monitoring. Sus valores superan 1 y llegan a 2,0, por lo que se interpretan como vCPU en uso y no como fracción. La línea de referencia de las figuras (1,2) corresponde al 60 % de 2 vCPU bajo esa interpretación. Las conclusiones de esta sección se apoyan en los eventos del MIG, que son directos, y no en el valor absoluto de esa serie.

3.5 Escenario 3B: autoscaling de workers

El 3B encola videos a una tasa fija de 20 por minuto durante 6 minutos (126 trabajos en total, incluida la rampa). Con WORKER_CONCURRENCY = 2, un worker procesa 2 trabajos a la vez y los tres juntos hasta 6.

Tabla 11. Escenario 3B: línea de tiempo de mig-workers y la cola

Hora UTC	Evento	Detalle
23:45:33	Inicio del encolado	20 trabajos por minuto hacia Redis
23:45:45	Scale-out de 1 a 3	El autoscaler fija el objetivo en 3 workers, 12 s después de empezar
23:46:06	3 VM creadas	Las 3 VM aparecen como estables en el MIG; los trabajos activos tardan más en llegar a 6
23:49:26	Primer momento con 6 trabajos activos	3 min 41 s después de la decisión: ya trabajan los 3 workers
23:51:28	Máximo de la cola	79 trabajos pendientes
23:51:33	Fin del encolado	El generador deja de enviar trabajos
00:00:52	Mayor antigüedad en cola	569 s (casi 10 minutos) con 7 pendientes
00:01:08	Cola de pendientes en 0	9 min 35 s después de terminar el encolado
00:01:53	Último trabajo activo	Fin real del drenado
00:12:51	Scale-in de 3 a 2	10 min 58 s después del último trabajo
00:13:11	Scale-in de 2 a 1	20 s después de la bajada anterior
00:13:53	mig-workers estable en 1	Fin del ciclo

Tabla 12. Escenario 3B: resultados

Métrica	Valor
Trabajos encolados	126 (20 por minuto)
Trabajos terminados	126 de 126, todos en estado ready
Máximo de pendientes	79
Máximo de activos en paralelo	6 (3 workers por concurrencia 2)
Antigüedad máxima en cola	569 s
Duración total del procesamiento	16 min 20 s, desde el primer encolado hasta el último trabajo activo
Ritmo de drenado con 3 workers	Unos 8 trabajos por minuto (79 pendientes en 9 min 40 s)
Respuestas de confirmación de la API	p50 85 ms, p95 121 ms, p99 211 ms, 0 % de errores en 882 peticiones
mig-api durante la prueba	Estable en 1 instancia: la API no se vio afectada por los workers
Cola al cerrar	Pendientes, activos, programados, reintentos y archivados en 0

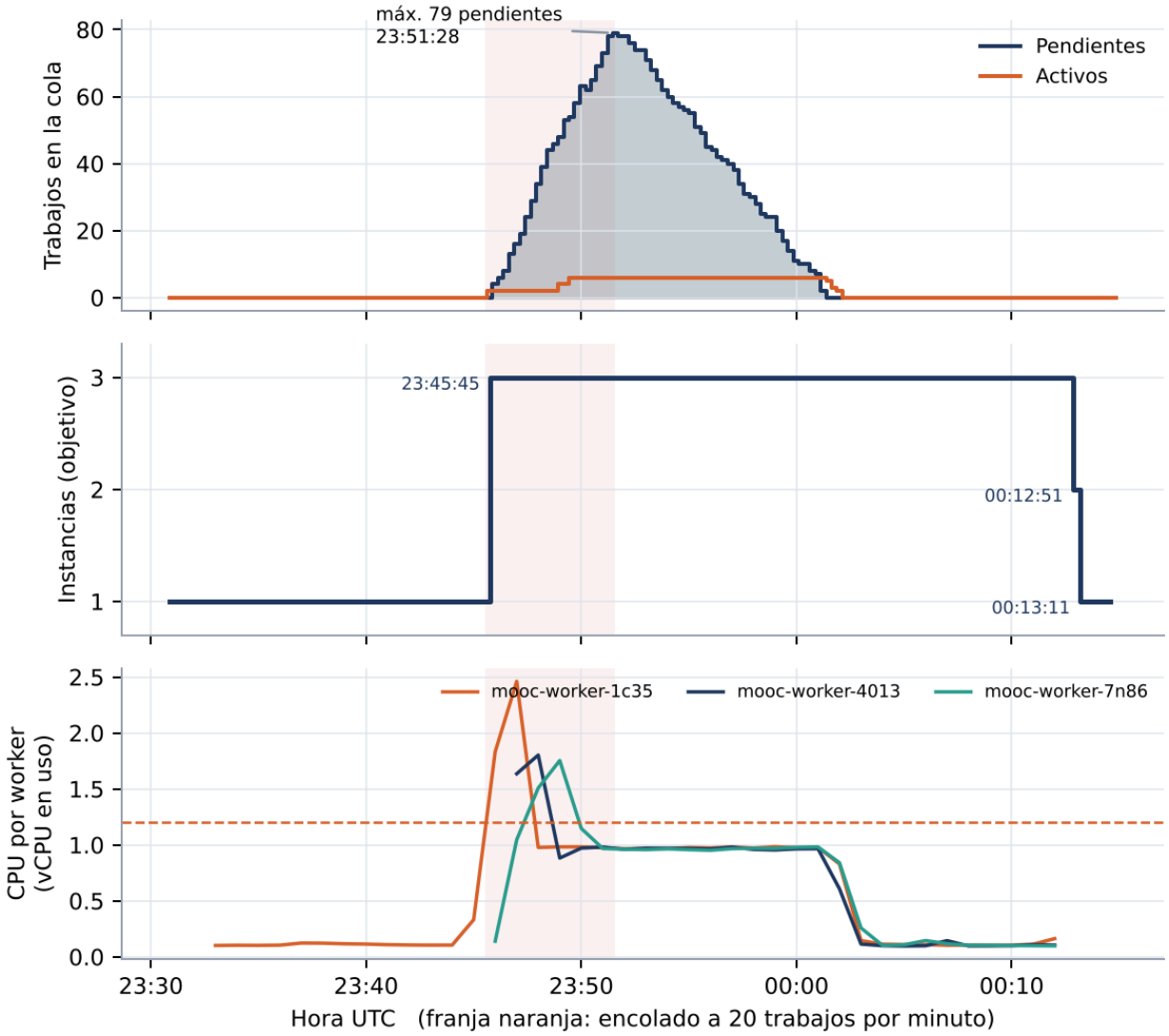


Figura 5. Escenario 3B: cola de Redis, tamaño de mig-workers y CPU de cada worker

La cola crece de forma lineal mientras se encola más rápido de lo que se procesa, llega a 79 pendientes y se vacía en paralelo cuando están los tres workers. La CPU de cada worker se sostiene cerca de 1 vCPU mientras hay trabajo y cae a nivel de reposo cuando la cola se vacía, y solo entonces empieza la cuenta regresiva del scale-in.

3.5.1 Integridad de los datos tras el escalado

Escalar workers puede duplicar trabajos o dejarlos a medias. Para descartarlo, al terminar la prueba se consultó la tabla de recursos multimedia en PostgreSQL desde una VM de la plataforma.

Tabla 13. Reconciliación SQL del escenario 3B

Consulta	Resultado
Activos del 3B por estado	126 en estado ready
Recursos con más de un activo (efecto duplicado)	0 filas
Activos sin estado final (perdidos o atascados)	0 filas
Fallos diagnosticables	0 filas

Los 126 trabajos terminaron una sola vez, sin duplicados, sin trabajos atascados y sin fallos. El detalle de la consulta está en formal-3b-1/reconciliacion.txt.

3.6 Scale-down de ambos grupos

Después de cesar la carga, los dos MIG vuelven a su mínimo sin intervención. GCP espera unos 10 minutos de CPU baja antes de retirar una instancia, lo que evita que el grupo oscile entre tamaños.

Tabla 14. Reducción de los grupos tras la carga

Corrida	Fin de la carga	3 a 2	2 a 1	Estable en 1
mig-api, escenario 1	01:10:04	01:20:42	01:21:25	01:23:53
mig-api, escenario 3A	23:02:53	23:13:54	23:23:57	23:26:04
mig-workers, escenario 2	02:33:14	02:45:15	02:45:57	02:46:38
mig-workers, escenario 3B	00:01:53 (cola vacía)	00:12:51	00:13:11	00:13:53

En mig-api la bajada de 3 a 2 y la de 2 a 1 estuvieron separadas por unos 10 minutos en el 3A. En mig-workers la segunda bajada llegó solo 20 a 42 segundos después de la primera, con la CPU ya en reposo. Por eso la reducción de los workers es más rápida que la de la API una vez que empieza.

3.7 Conclusiones, limitaciones y trabajo pendiente

- **Capa web.** El balanceador con mig-api elimina el cuello de botella de la máquina fija: p95 entre 97 % y 98 % más bajo, 0,0005 % de errores y 45 % más peticiones por segundo.
- **Autoscaling de la API.** mig-api pasó de 1 a 3 instancias bajo una ráfaga de 250 usuarios virtuales (la tercera se pidió 20 segundos después de la segunda), sin errores, y volvió a 1 por sí sola.
- **Autoscaling de workers.** mig-workers pasó de 1 a 3 instancias, drenó 126 trabajos en paralelo, sin duplicados ni pérdidas, y volvió a 1.
- **Siguiente cuello.** Cloud SQL llegó a cerca del 75 % de CPU y el pool de conexiones a su tope de 60 con 3 instancias de API.

Limitaciones. Cada escenario tiene una sola corrida formal, así que no se mide la variación entre repeticiones. El escalado tarda de 3 a 4 minutos en dar capacidad, por lo que durante ese tiempo la latencia de cola es mayor. La línea base de la Entrega 2 para el escenario 2 se completa con la tabla oficial de esa entrega. La unidad de las series de CPU por VM debe confirmarse en la consola de Cloud Monitoring.

Trabajo pendiente. Repetir cada escenario para reportar la variación entre corridas, investigar el costo fijo del login, evaluar una señal de escalado por profundidad de cola para los workers, y subir la capacidad de Cloud SQL antes de aumentar el máximo de mig-api.

3.8 Evidencias y reproducibilidad

Todas las evidencias están en load/salida/, una carpeta por corrida. Cada una contiene el reporte de la corrida, el resumen de k6, las series de Cloud Monitoring, las series del observador y, cuando aplica, la reconciliación SQL.

Tabla 15. Archivos de evidencia por corrida

Archivo	Contenido
corrida-escenario*.md	Parámetros, commit, URL, generador y horas de inicio y fin
resumen-escenario*.json	Resumen de k6: peticiones, latencias, checks, umbrales y marcas de fase
series-formal-*.csv	Series de Cloud Monitoring por VM y por Cloud SQL (CPU, memoria, red, disco, conexiones) y peticiones del balanceador
serie-formal-*-mig.csv	Tamaño objetivo, recomendado y estable de mig-api y mig-workers cada pocos segundos
serie-formal-*-cola.csv	Profundidad de la cola de Redis (pending, active, scheduled, retry, archived) y antigüedad del trabajo más viejo
metricas-formal-*.md y .json	Resumen estadístico (mediana, p95, p99 y máximo) de cada serie
reconciliacion.txt (3B)	Consultas SQL de integridad sobre los activos multimedia
NOTA.txt (intento descartado)	Causa por la que el primer intento del escenario 2 no cuenta

Las figuras de esta sección se generaron a partir de esos archivos CSV, y los eventos de escalado se tomaron de las series del observador.

4. Calidad de código, deuda técnica y retiro seguro

Esta sección presenta el primer diagnóstico formal de deuda técnica del proyecto. El reporte de SonarQube del commit 83eb29e y el inventario asociado constituyen la línea base para las siguientes entregas. El detalle completo está en docs/entrega3/deuda-tecnica.md, y el reporte exportado en docs/entrega3/evidencias/sonar/.

4.1 Alcance y método de diagnóstico

El análisis corre en GitHub Actions en cada push, con el flujo .github/workflows/sonar.yml. Además, el equipo revisó manualmente el código de cola y retiro, la infraestructura, los scripts de carga y la automatización existente, porque esos riesgos no siempre aparecen en un análisis estático.

Tabla 16. Configuración del análisis

Campo	Valor
Herramienta	SonarQube 10.7 Community y SonarScanner CLI 11.1
Perfiles de reglas	Sonar way para Go y TypeScript, sin reglas desactivadas
Puerta de calidad	Duplicación máxima de 20 % y mantenibilidad C o mejor, sobre todo el código
Código analizado	API, workers y frontend
Exclusiones	Dependencias, salidas de compilación, pruebas y sus ayudas, estáticos y archivos generados. No se excluye código de aplicación
Commit evaluado	83eb29e
Fecha	1 de noviembre de 2026
Pipeline	SonarQube #70 y CI #141, ambos exitosos
Reporte	docs/entrega3/evidencias/sonar/resumen.md

4.2 Resumen de la medición

Tabla 17. Medición inicial y línea base

Métrica	Inicial (31d5af0)	Línea base (83eb29e)	Umbral
Duplicación	5,2 %	0,4 %	Máximo 20 %
Mantenibilidad	A	A	C o mejor
Confiabilidad	A	A	Sin umbral
Seguridad	E	A	Sin umbral
Bugs	0	0	Sin umbral
Vulnerabilidades	1	0	Sin umbral
Deuda técnica estimada	963 min	1.152 min	Registrar
Ratio de deuda	0,2 %	0,2 %	Sin umbral
Code smells	156	196	Sin umbral
Security hotspots	6	9	Sin umbral
Cobertura	11,8 %	30,1 %	Sin umbral
Líneas de código	18.344	19.780	Sin umbral

La deuda de 1.152 minutos equivale a unas 19 horas de esfuerzo estimado. Es la primera medición de referencia. Subió frente a la inicial porque creció el código de frontend, aunque el ratio se mantuvo en 0,2 %.
Esa cifra debe leerse junto con el inventario: mide principalmente estilo y complejidad, no riesgo operativo. El defecto más grave encontrado, la pérdida de trabajos ante la caída de un worker, no aparecía en esa estimación.
La cobertura subió porque el análisis ahora ejecuta pruebas de integración que antes se omitían. No refleja una mejora equivalente en pruebas nuevas, y el frontend todavía no reporta cobertura.

Tabla 18. Hallazgos abiertos en la línea base

Severidad	Cantidad
Bloqueante	0
Crítico	38
Mayor	119
Menor	12
Informativo	27
Total	**196**

4.3 Correcciones realizadas

Tabla 19. Correcciones de esta entrega

Corrección	Problema	Resultado	Commit
Catálogos de idioma a JSON	Dos bloques casi iguales de 540 líneas	Principal aporte a la reducción de duplicación	98a6bb8
Esqueleto común de listas personales	Estados de carga, vacío y error repetidos	Se deciden en un solo componente	98a6bb8
Retiro seguro con SIGTERM	El siguiente worker descartaba el trabajo devuelto	El trabajo vuelve a la cola sin gastar reintento	98a6bb8
Clave retirada del código	DATABASE_URL tenía una clave por defecto	Seguridad pasa de E a A	e07e872
Recuperación tras caída abrupta	La reentrega se daba por terminada y el recurso quedaba atascado	La entrega se aplaza hasta vencer el arrendamiento	ca0760b
CI operativo	La imagen de MinIO ya no se podía descargar, y otros fallos quedaban ocultos	CI #141 en verde	e1765a8, 746f11d y siguientes
Bug y selector duplicado	typescript:S5850 y css:S4666	Confiabilidad vuelve a A	83eb29e

4.4 Inventario priorizado de deuda técnica

El inventario completo, con identificador, ubicación, evidencia, impacto, prioridad, tratamiento y estado, está en la sección 4 de docs/entrega3/deuda-tecnica.md. Cada hallazgo Mayor o superior tiene su archivo y línea en el reporte exportado.

La prioridad P0 corresponde a requisitos de esta entrega, P1 a riesgos reales, P2 a mantenimiento y P3 a estilo o limpieza.

Tabla 20. Principales elementos del inventario

ID	Hallazgo	Impacto	Prioridad	Estado
DT-01	Workers escalan por CPU, no por demanda de cola	Puede escalar con la cola vacía y retirar instancias ocupadas	P0	En tratamiento
DT-02	Registros solo en la VM	Se pierden al retirar una instancia	P0	En tratamiento
DT-03	Plantillas con etiqueta móvil	Una instancia puede correr otra versión	P0	En tratamiento
DT-04	Arranque mayor que el periodo del autoscaler	La CPU de arranque influye en las decisiones	P0	En tratamiento
DT-05	/healthz no consulta dependencias	Una API sin base puede seguir recibiendo tráfico	P0	En tratamiento
DT-06	Migraciones al arrancar cada réplica	Un fallo aparece como réplicas que no arrancan	P0	En tratamiento
DT-19	Contenedores como root	Mayor impacto si FFmpeg o LibreOffice fueran comprometidos	P1	Aplazado a Entrega 4
DT-13	Puntos únicos de falla en Redis y Cloud SQL	Una caída detiene cola o API	P2	Aceptado temporalmente
DT-14	Redis sin contraseña	Control de acceso solo por red	P2	Aceptado temporalmente
DT-17	MinIO desde fork comunitario	Dependencia de un tercero en local y CI	P3	Aceptado temporalmente

Hotspots. Se revisaron los 9. Siete no representan riesgo en su contexto, y dos confirman que los contenedores corren como root, lo que se registra en DT-19.

4.5 Plan de tratamiento

Los hallazgos P0 son requisitos de la entrega y deben cerrarse antes del cierre: registrarlos no los subsana.

Los 38 hallazgos Críticos son de mantenibilidad y no tienen bugs asociados. Se aplazan porque modificar funciones centrales justo antes de las pruebas añadía más riesgo que beneficio.

Tabla 21. Plan para los hallazgos Críticos

Plan	Hallazgos	Pasos	Responsable	Plazo
C1	Complejidad cognitiva y anidación (22 casos)	Ordenar por complejidad, añadir pruebas de caracterización y partir una función por commit	Calidad y Frontend	Primera semana de la Entrega 4
C2	Contadores modificados y literales repetidos (16 casos)	Extraer constantes y reescribir recorridos con las pruebas de ida y vuelta como respaldo	Calidad y Frontend	Segunda semana de la Entrega 4

4.6 Evidencia de retiro seguro de workers

La comprobación se ejecutó en GCP, con dos workers, Redis y Cloud SQL. Con FFmpeg en curso, se detuvo el worker con docker stop, que envía SIGTERM. El worker dejó de tomar trabajos, canceló FFmpeg y devolvió sus trabajos a la cola sin consumir reintentos. Otro worker tomó el video y lo reinició desde el original, que permaneció intacto: se pierde el avance, no el trabajo aceptado.

Tabla 22. Resultado de la comprobación

Elemento	Resultado
Fecha	1 de noviembre de 2026, 15:09 UTC
Commit	746f11d
Worker retirado	mooc-worker-cmkb
Tiempo del retiro	14 s
Código de salida	0, sin SIGKILL
Worker que retoma	mooc-worker-c5f7, en menos de 2 s
Video listo	15:26:37 UTC, unos 17 minutos después
Finalizaciones	1
Estado en Cloud SQL	Activo y recurso en ready; sin activos sin terminar ni desacuerdos
Cola	pending 0, active 0, retry 0, archived 1

El trabajo archivado corresponde al segundo activo que tenía el worker. Su registro ya no existía en la base y el error quedó diagnosticado en la dead-letter queue (DT-21).

Esta comprobación complementa la reducción automática del escenario 3B y no la reemplaza. Si un worker muere sin SIGTERM, la entrega se aplaza hasta que vence su arrendamiento de 30 minutos y otro worker la toma. Ese caso está cubierto por pruebas de integración contra PostgreSQL y Redis.

Evidencia: docs/entrega3/evidencias/retiro-seguro/contenedor-20261101T150752Z/, deploy/retiro-seguro/demo-gcp.sh y deploy/retiro-seguro/verificar.sql